

Honest Cross-Validation Selects Models That Deploy Better on New Households: Selection Regret in UK Smart-Meter Demand Forecasting

Evgenii Lazarev^{1,*}

¹Independent Researcher

*elazarev@gmail.com

ORCID: 0009-0000-1398-7842

Abstract

Cross-validation does two jobs at once. It estimates out-of-sample error, and it ranks candidate models so that the top-ranked one is the model that ships. The second job is rarely audited. On panel data with repeated entities, shuffled k-fold can rank a model that memorizes entity identity above a simpler model that actually generalizes, because the test fold contains rows from the same entities as the training fold. The selected model then underperforms on entities it has never seen. We measure this effect on a real, partially predictable task: next-day household electricity demand from the UK Low Carbon London smart-meter trial. We select a model from a fixed eight-candidate grid (k-nearest neighbors, Ridge, and Random Forest) twice, once under naive shuffled k-fold and once under group-aware purged cross-validation, then deploy each selected model on twelve households held out entirely from selection. Naive cross-validation selects an unconstrained Random Forest that splits on a household-identifier feature and deploys at 2.638 kWh mean absolute error; group-aware cross-validation selects Ridge and deploys at 2.464 kWh, a 6.6% reduction with higher coefficient of determination. Across 30 random household partitions the group-aware selection deploys better in every partition, with a mean deployment-regret reduction of 0.53 kWh (95% confidence interval 0.34 to 0.73). Removing the household-identifier feature collapses both selections onto the same model and erases the gap, isolating entity leakage as the mechanism. The practical message is that the cross-validation unit should match the deployment unit: forecasting for new customers calls for holding out customers, not rows.

Keywords: cross-validation; model selection; data leakage; load forecasting; smart meters; generalization

1 Introduction

Cross-validation is usually described as a way to estimate how well a model will generalize. In practice it carries a second responsibility that receives less scrutiny: it is the procedure that picks which model gets deployed. An analyst fits a grid of candidates, scores each by cross-validated error, and ships the one with the best score. If the cross-validation score is a faithful proxy for deployment error, the two jobs coincide and the best-scoring model is also the best deploying model. When the proxy is biased in a way that favors one model class over another, the two jobs diverge, and the analyst can ship a model that scored well for the wrong reason.

This divergence has a name. Teodorescu and Obreja Braşoveanu [1] formalize *model-selection regret* as the loss in out-of-sample performance from choosing the model with the best cross-validation score rather than the model with the best out-of-sample score. Studying bankruptcy prediction on independent and identically distributed tabular data, they find that k-fold cross-validation is a valid selection rule on average, but that roughly 67% of the variance in regret is attributable to the particular train and test split rather than to the selection procedure, which they read as an irreducible uncertainty that more elaborate cross-validation cannot remove. They also report a more pointed result and leave it open: decisions *between model classes* taken on cross-validation performance are, more often than not, incorrect, and this warrants further investigation. The present paper takes up exactly that open question, in a setting where the data are not independent and identically distributed but grouped and time-indexed, and where the regret source is not irreducible but a removable form of leakage.

Leakage is a well-documented cause of inflated performance estimates in machine learning [2], and it is especially costly when a model is selected and reported after many iterations, so that the optimistic score

becomes part of the evidence rather than a private development error [3]. In financial machine learning, López de Prado [4] introduced purging and embargoing to guard against leakage from overlapping labels and serial dependence, and group-disjoint folds to evaluate generalization to unseen entities. Recent work continues to document how validation design alone can swing reported accuracy in time-series forecasting [5]. Most of this literature measures leakage as *bias in the score*. We measure something operationally sharper: whether leakage changes *which model is selected*, and what that wrong selection costs at deployment.

Smart-meter demand forecasting is a natural place to study this. The data are a panel: many households, each observed over time. A common deployment target is a new customer with little or no history, for whom the relevant generalization unit is the household, not the calendar row. Yet the convenient default, shuffled k-fold, scatters a household’s rows across training and test folds, so a model can learn a per-household baseline during selection and look excellent, then fail on customers it never saw.

This paper makes four contributions. First, we give a concrete demonstration on real energy data that the cross-validation scheme changes the deployed model class, not merely the reported error. Second, we quantify the deployment-regret reduction from group-aware selection on truly held-out households, with confidence intervals over 30 random partitions. Third, we isolate the mechanism with an ablation that removes a single entity-identifier feature and watches the gap disappear. Fourth, the entire study is reproducible from open code: the splitter is the `scikit-learn-compatible` `purgedcv` package [6, 7], and the experiment is a single script over a public dataset.

2 Materials and Methods

2.1 Data

We use the Low Carbon London smart-meter dataset released by UK Power Networks through the London Datastore [8]. The analysis uses a cached 60-household subset of half-hourly consumption, restricted to the 2013 calendar year and aggregated to daily consumption per household in kWh. The prediction target is a household’s daily consumption; the task is partially predictable, because daily load has strong day-of-week and short-lag structure but also large idiosyncratic variation.

Households are the unit of deployment. For each random seed we permute the 60 households and assign 48 to a selection set and the remaining 12 to a deployment set that is never seen during model selection. With seed 0 the selection set contains 16,090 daily observations from 48 households and the deployment set contains 4,122 observations from 12 held-out households.

2.2 Features

Each row carries eleven features: the previous day’s consumption (`lag_1`), the same weekday one week earlier (`lag_7`), a one-hot encoding of the day of week (`dow_0` through `dow_6`), and the household identifier mapped to an integer (`hh_int`). The identifier is included on purpose. Inside shuffled k-fold, where the same household appears in both training and test folds, a sufficiently flexible model can split on `hh_int` and memorize each household’s baseline level, which lowers the cross-validation error without improving generalization. On a genuinely unseen household the identifier carries no information. Identifier-like features of this kind are common in applied pipelines, which is why the trap is realistic rather than contrived.

2.3 Candidate grid

The candidate grid spans eight models from overfitting-prone to constrained: k-nearest neighbors with $k \in \{1, 5, 50\}$, Ridge regression with $\alpha \in \{0.01, 1, 100\}$, and Random Forest regressors with `max_depth` of `None` (unconstrained, free to split deeply on `hh_int`) and 4 (shallow). The k-nearest-neighbors and Ridge candidates are standardized through a `scikit-learn` pipeline. The grid is fixed in advance and identical across both cross-validation schemes, so any difference in the selected model is caused by the validation design and nothing else.

2.4 Cross-validation schemes

Two five-fold schemes are compared. The naive scheme is `KFold(n_splits=5, shuffle=True)`, which ignores household identity and scatters rows at random, so training and test folds routinely share

households. The honest scheme is `PurgedGroupKfold` from `purgedcv`, which keeps whole households together, so every test fold contains households absent from the corresponding training fold. This mirrors the deployment condition exactly. Because the labels here are instantaneous daily values rather than forward-horizon intervals, we set the evaluation time equal to the prediction time, so the group-aware splitter performs a pure household-disjoint split with no time-purging. This isolates the group-leakage effect from any overlapping-label effect.

2.5 Selection-regret protocol

Following Teodorescu and Obreja Braşoveanu [1], define regret as

$$\delta = \text{OOS}(M^{\text{OOS}}) - \text{OOS}(M^{\text{CV}}), \quad (1)$$

the deployment-error gap between the model that is actually best out of sample, M^{OOS} , and the model selected by cross-validation, M^{CV} . Here the out-of-sample measurement is deployment on the 12 held-out households. Our reported quantity is the difference in this deployment error between the two selectors: the naive deployment mean absolute error minus the honest deployment mean absolute error. A positive value means the honest selector deployed better.

The protocol is identical for both schemes. Each candidate is scored by five-fold cross-validated mean absolute error on the selection set. The lowest-error candidate is selected, retrained on the full selection set, and evaluated on the deployment households by mean absolute error and coefficient of determination R^2 . To move beyond a single realization, the whole pipeline, including the household partition, the shuffle, and the Random Forest seeds, is repeated for 30 seeds, and we report the distribution of the deployment-regret difference with a 95% t-interval and the fraction of seeds in which the honest selector wins.

2.6 Ablation

To test that the household identifier is the mechanism, the experiment is repeated with `hh_int` removed from the feature set. If the identifier was the only route to leakage, both selectors should now pick the same model and the deployment gap should vanish.

2.7 Computational environment

The reported numbers were produced with `purgedcv` 0.0.11, Python 3.12.7, `numpy` 1.26.4, `pandas` 2.3.3, `scikit-learn` 1.8.0, and `scipy` 1.17.1 on macOS 26.3.1 (arm64). The full study is a single script, `examples/selection_regret_lcl_seeds.py`, and a companion notebook, `examples/selection_regret_lcl.ipynb`, that reproduces the single-seed walk through and the headline figure.

3 Results

3.1 The two schemes select different model classes

Table 1 shows the five-fold cross-validated mean absolute error of every candidate under both schemes, for seed 0. Under naive shuffled k-fold the unconstrained Random Forest posts the lowest error, 1.6520 kWh, because it can read each household’s baseline off the identifier feature and the same households appear in the test fold. Under group-aware cross-validation that advantage disappears: the unconstrained Random Forest rises to 2.0018 kWh, and the lowest-error candidate is Ridge with $\alpha = 0.01$ at 1.7094 kWh. The two schemes therefore select different model *classes* from the same grid on the same data.

3.2 The honest selection deploys better on unseen households

Table 2 reports what happens when each selected model is retrained on the full selection set and evaluated on the 12 held-out households. The Random Forest chosen by naive cross-validation deploys at 2.6376 kWh mean absolute error, while the Ridge model chosen by group-aware cross-validation deploys at 2.4641 kWh. The honest selection is 0.1735 kWh lower, a 6.6% reduction in deployment error, and it also has the higher coefficient of determination (0.7834 versus 0.7587). Figure 1 shows the contrast.

Table 1: Five-fold cross-validated mean absolute error (kWh) per candidate on the selection set, seed 0. The selected model in each column is the row with the lowest error.

Candidate	Naive shuffled k-fold	Group-aware (honest)
kNN ($k = 1$)	2.1175	2.7864
kNN ($k = 5$)	1.7413	2.2254
kNN ($k = 50$)	1.7603	2.2544
Ridge ($\alpha = 0.01$)	1.6913	1.7094
Ridge ($\alpha = 1$)	1.6914	1.7095
Ridge ($\alpha = 100$)	1.6948	1.7136
RF (<code>max_depth=None</code>)	1.6520	2.0018
RF (<code>max_depth=4</code>)	1.7197	1.8649
Selected	RF (<code>max_depth=None</code>)	Ridge ($\alpha = 0.01$)

Table 2: Deployment on 12 held-out households, seed 0. Lower mean absolute error and higher R^2 are better.

Selector	Selected model	Deployment MAE (kWh)	Deployment R^2
Naive	RF (<code>max_depth=None</code>)	2.6376	0.7587
Honest	Ridge ($\alpha = 0.01$)	2.4641	0.7834
Difference (naive minus honest)		+0.1735 (+6.6%)	−0.0246

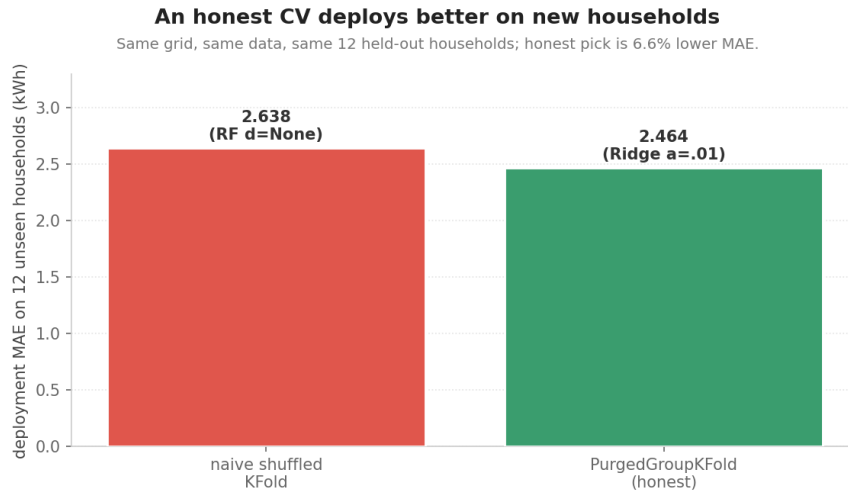


Figure 1: Deployment mean absolute error on the 12 held-out households for the model each scheme selects, seed 0. Same grid, same data, same held-out households; the honest selection deploys at 6.6% lower error.

3.3 The advantage holds across 30 random partitions

A single household partition is one draw, and Teodorescu and Obreja Braşoveanu [1] warn that the partition itself is the dominant source of regret variance. We therefore repeat the full pipeline over 30 seeds. The selection is strikingly stable: naive cross-validation selects the unconstrained Random Forest in all 30 seeds, and group-aware cross-validation selects Ridge with $\alpha = 0.01$ in all 30 seeds. The honest selection deploys at lower mean absolute error in every one of the 30 partitions. The deployment-regret difference of Equation (1) has a mean of +0.5319 kWh with a 95% confidence interval of +0.3376 to +0.7261 kWh, a median of +0.2831 kWh, and a range from +0.1084 to +1.7851 kWh. In relative terms the mean reduction is 19.8% (95% confidence interval 14.1% to 25.5%) with a median of 13.3%. Table 3 summarizes, and Figure 2 shows the paired deployment errors and the distribution of the regret difference. The distribution lies entirely on the positive side of zero: there is no seed in which naive selection deployed better.

Table 3: Robustness over 30 random 48/12 household partitions. The deployment-regret difference is naive deployment MAE minus honest deployment MAE; positive favors the honest selector.

Quantity	Value
Naive selects RF (<code>max_depth=None</code>)	30 / 30 seeds
Honest selects Ridge ($\alpha = 0.01$)	30 / 30 seeds
Honest deploys better	30 / 30 seeds
Mean regret difference (kWh)	+0.5319
95% confidence interval (kWh)	+0.3376 to +0.7261
Median regret difference (kWh)	+0.2831
Range (kWh)	+0.1084 to +1.7851
Mean relative reduction	19.8%
95% confidence interval	14.1% to 25.5%
Median relative reduction	13.3%

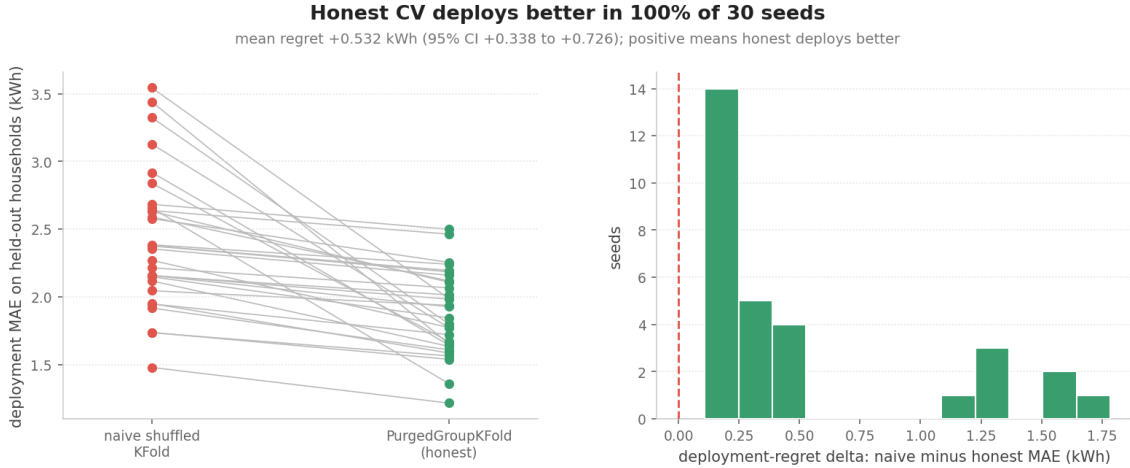


Figure 2: Left: deployment mean absolute error for the naive and honest selections in each of the 30 seeds; every paired line falls. Right: distribution of the deployment-regret difference across seeds; all of the mass is positive, meaning the honest selection deploys better in every partition.

3.4 Removing the identifier erases the gap

When the household identifier `hh_int` is removed from the feature set, both schemes select Ridge with $\alpha = 0.01$, and both deploy at the same mean absolute error of 2.4676 kWh on the held-out households. The regret difference falls to zero. This pins the mechanism precisely: the gap was not a property of Random Forests in general, nor of the cross-validation arithmetic, but of one entity-identifier feature that shuffled cross-validation allowed a flexible model to memorize. Group-aware cross-validation refuses that information during selection, and the selection then matches what deployment will look like.

3.5 Household structure carries a large generalization penalty at scale

The single-grid study above uses 60 households. A full-population benchmark on the same corpus confirms that the household structure carries a substantial generalization penalty well beyond this sample. Scanning 167,932,474 raw rows and drawing 20 seeded subsamples of 60 households, scoring on entirely unseen households is 6.03% worse in weighted absolute percentage error than a pooled temporal estimate (95% confidence interval 4.93% to 7.12%), whereas the purely temporal leakage gap is only 1.60%. The dominant operational risk in this corpus is therefore household generalization rather than temporal leakage, which is exactly the risk that group-aware selection is built to measure.

4 Discussion

4.1 Mechanism

The result has a simple cause. Shuffled cross-validation lets a model see, during scoring, information it will not have at deployment time, namely the identity and baseline level of households that will also appear in the test fold. A deep Random Forest exploits this by learning per-household leaf values, which makes its cross-validation error look the best in the grid. On households it has never seen, the identifier is meaningless and the memorized structure is dead weight, so the deep forest falls behind a regularized linear model. Group-aware cross-validation removes the leaked information during selection, so its ranking reflects the deployment condition and it selects the model that actually generalizes.

4.2 Relation to prior work

Teodorescu and Obreja Braşoveanu [1] conclude that on independent and identically distributed tabular data the train and test split dominates regret variance, so more elaborate cross-validation buys little on average. Our finding is not a contradiction but a complement that depends on the data regime. On their i.i.d. data the regret source is largely irreducible. On grouped, time-indexed data the regret source is reducible: it is entity leakage, and a validation scheme matched to the deployment unit removes it. We also answer their explicit open question. They observe that decisions between model *classes* taken on cross-validation performance are often wrong and call for further investigation; here a cross-class decision (Ridge versus Random Forest) is wrong under naive cross-validation in every one of 30 partitions and right under group-aware cross-validation in every one of them. The lesson generalizes beyond households to any repeated-entity panel, including patients, devices, stores, and sensors.

4.3 Practical guidance

The actionable rule is to match the cross-validation unit to the deployment unit. If a demand forecaster will serve customers who were present during training, a pooled or temporal split may be appropriate. If it will serve new customers, selection must hold out customers, because that is the question deployment will ask. The choice is not a tuning detail; it determines which model class is shipped. The `purgedcv` package makes the group-aware split a drop-in `scikit-learn` cross-validator, so adopting the honest selection costs one line of code.

4.4 Limitations

The study uses one dataset, one forecasting target at daily resolution, and a deliberately compact grid; the magnitudes will differ on other corpora, horizons, and richer grids. The household identifier is an intentional trap, but identifier-like and entity-correlated features are common in real pipelines, and the ablation shows the effect is driven by exactly such a feature rather than by an artifact of the splitter. The robustness sweep covers 30 partitions of a 60-household sample; the population-scale benchmark corroborates the direction and size of the household penalty but uses a different scoring metric and a single estimator. Finally, group-aware validation addresses entity leakage; it does not by itself remove other leakage routes such as globally fit preprocessing or future-looking feature engineering.

5 Conclusions

On a real, partially predictable demand-forecasting task, the cross-validation scheme used for model selection decided which model class was deployed. Naive shuffled k-fold consistently selected an unconstrained Random Forest that memorized household identity and deployed worse on new households, while group-aware purged cross-validation consistently selected Ridge and deployed better, by 6.6% on the headline split and in every one of 30 random partitions. The effect is removable, not irreducible: it is entity leakage, and a validation unit matched to the deployment unit removes it. For demand forecasting aimed at new customers, and for panel forecasting more broadly, validation should be performed at the granularity of the entity the model will actually face.

Data availability

The Low Carbon London smart-meter data are publicly available from UK Power Networks and the London Datastore [8]. The repository includes the cached 60-household subset used here, the experiment script, the seed-sweep output, and the notebook needed to reproduce every reported number and figure.

Code availability

The cross-validation splitter is provided by the open-source `purgedcv` package, available at <https://github.com/eslazarev/purged-cross-validation> and on PyPI as `purgedcv` [6]. The software is MIT licensed and archived on Zenodo with DOI 10.5281/zenodo.20312695. The experiment in this paper is reproduced by `examples/selection_regret_lcl_seeds.py` and `examples/selection_regret_lcl.ipynb`.

Use of generative AI tools

Large language model tools were used as assistants for code review, refactoring suggestions, copy-editing, and pre-submission checks. They were not treated as authors. All design decisions and outputs were reviewed and validated by the author; no numerical claim was accepted without executable verification against the experiment script.

References

- [1] Vlad Teodorescu and Laura Obreja Braşoveanu. Assessing the validity of k-fold cross-validation for model selection: Evidence from bankruptcy prediction using random forest and xgboost. *Computation*, 13(5):127, 2025. 10.3390/computation13050127.
- [2] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4): 1–21, 2012. 10.1145/2382577.2382579.
- [3] Matthew B. A. McDermott, Shirly Wang, Nikki Marinsek, Rajesh Ranganath, Luca Foschini, and Marzyeh Ghassemi. Reproducibility in machine learning for health research: Still a ways to go. *Science Translational Medicine*, 13(586), 2021. 10.1126/scitranslmed.abb1655.
- [4] Marcos López de Prado. *Advances in Financial Machine Learning*. Wiley, Hoboken, NJ, 2018. ISBN 9781119482086. Purging and embargoing: chapter 7. Combinatorial Purged Cross-Validation: chapter 12.
- [5] Salma Albelali and Moataz Ahmed. Hidden leaks in time series forecasting: How data leakage affects lstm evaluation across configurations and validation strategies, 2025.
- [6] Evgenii Lazarev. `purgedcv`: scikit-learn-compatible purged and combinatorial cross-validation for time-series and panel machine learning. Python package, <https://github.com/eslazarev/purged-cross-validation>, 2026. Software concept DOI; MIT license.

- [7] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [8] UK Power Networks. Smartmeter energy consumption data in london households (low carbon london). London Datastore, 2014. Dataset `smartmeter-energy-use-data-in-london-households`; open terms.

Author contributions

E.L. conceived the study, implemented the software, designed and ran the experiments, analyzed the results, wrote the manuscript, and approved the final version.

Funding

No external funding was received for this work.

Conflicts of interest

The author declares no conflict of interest.